STANDARD OPERATING PROCEDURE

Gene list to Stocker stock sheets

This PDF is enough to install Stocker from GitHub, understand what it does, and run the lab default. The output is only as good as the gene list, the config you leave in place, and the stock limits that config applies.

Audience	Default config	Validate IDs
Anyone installing or running Stocker for the first time	priority_example.json	flybase.org/convert/id

What this document covers

Three decisions determine whether the workbook is usable. Everything else (install, commands, troubleshooting) exists so those three decisions are done correctly.

1. **The gene list is solid.** Every symbol maps to the intended current *D. melanogaster* FlyBase gene. Wrong FBgn IDs silently query the wrong genes.
2. **The default config is understood.** The JSON file is the sorting protocol. It does not add genes or stocks. Changing a keyword or a sheet order changes the science.
3. **Stock limits are understood.** The default keeps at most 5 stocks per gene and 3 per allele, counted across the whole workbook. Higher-priority sheets spend the quota first.

Copy-paste URLs

```
Project (GitHub):     https://github.com/aadish98/fl.AI-reagent-stocker
ID validator:         https://flybase.org/convert/id
Gene report example:  https://flybase.org/reports/FBgn0023076
Python:               https://www.python.org/downloads/
Cursor (optional):    https://cursor.com
```

Every FBgn in the review workbook uses this report pattern: `https://flybase.org/reports/FBgn########` (example Clock: `https://flybase.org/reports/FBgn0023076`).

1. How Stocker works

Stocker is a *Drosophila* reagent-lookup tool. You give it gene symbols. It returns the FlyBase stocks that perturb those genes, together with the papers already attached to those stocks, sorted into experimental categories.

It does not invent stocks, call DEGs, or decide that a knockdown "worked." It organizes records that already exist in a local FlyBase snapshot and in PubMed titles and abstracts. Availability and stock health still have to be checked at BDSC, VDRC, or the relevant center before you order.

The path, in biological order:

1. **Identity.** Each symbol is matched to one current fly FBgn (for example Clock FBgn0023076).
2. **Reagents.** Alleles, constructs, and insertions of that FBgn are collected.
3. **Stocks.** Orderable (and some custom) genotypes at BDSC, VDRC, and other centers.

4. **Literature and phenotypes.** FlyBase citations are attached; PubMed supplies title and abstract. Curated FlyBase phenotypes are scored against the config's phenotype words.
5. **Sorting.** Each stock is tested against the config's category list, in order, and kept in the **first** category it qualifies for.
6. **Limits.** Across the whole workbook, at most 5 stocks per gene and 3 per allele are kept. Leftovers that hit the cap are dropped, not spilled to later sheets.

One gene-list CSV becomes one workbook. Two lists (for example two DEG contrasts) become two independent workbooks. You provide the list and the config. Stocker consults FlyBase and PubMed. You receive sorted sheets plus a counts table.

2. One-time install from GitHub

You need Python 3 and git. Every later command runs from the repo root (`fl.AI-reagent-stocker`), not from a Downloads folder or a PDF-only Cursor project. If the GitHub repo is private, ask the maintainer for access first. Source: `https://github.com/aadish98/fl.AI-reagent-stocker`.

A. Clone and install Python packages. `python` means Python 3; use `python3` if `python` is missing.

```
git clone https://github.com/aadish98/fl.AI-reagent-stocker.git
cd fl.AI-reagent-stocker
python3 --version
python3 -m pip install -r requirements.txt
```

B. Download the FlyBase citation table. A GitHub clone includes most stock, allele, phenotype, and synonym tables, plus the helper files `fbst_to_derived_stock_component.csv` and `fbtp_to_fbti.csv`. The large citation file `data/flybase/references/entity_publication_fb_*.tsv.gz` is not in git. Without it, Stocker cannot attach papers. Run this once after clone (and again when you want a newer FlyBase release):

```
python3 scripts/refresh_flybase_data.py
```

That command pulls the current FlyBase TSV families into `data/flybase/`. You do **not** need `--with-xml` for a first run. The XML flag is only for rebuilding the helper CSVs, which already ship in the repo.

C. Optional keys in a repo-root `.env`. The default config turns **embeddings on** and **GPT validation off**. Stock sheets still build without a key. Cosine-similarity columns need `OPENAI_API_KEY`. `NCBI_API_KEY` only raises PubMed rate limits.

```
OPENAI_API_KEY=sk-...
NCBI_API_KEY=...
```

Confirm `data`, `scripts`, `fl_ai_reagent_stocker`, and `data/config/stock_split_config_priority_example.json` are visible. If you were handed a complete folder on a shared drive, skip clone; still run the pip install and confirm `entity_publication` exists under `data/flybase/references/`.

Optional: open that same repo folder in Cursor and paste the commands below into Agent chat or **View > Terminal**. Cursor itself is at `https://cursor.com`. The agent must run every Python command from the repo root.

3. Make the gene list solid

This is the hard gate. Do not run Stocker until a human has reviewed the FBgn IDs. A clean install and a correct config cannot rescue a list that points at the wrong genes.

> **NON-NEGOTIABLE · HUMANS AND AGENTS**
> **Wrong FBgn IDs silently send Stocker at the wrong genes. Conversion must always go through `scripts/fetch_fbgn_ids.py`.**
> Never edit the user-generated gene list. The script writes new files (`validated_<original>.csv`, `needs-review.csv`, and `validated_<original>.xlsx`). It does not overwrite the original.
> Never invent, recall, autocomplete, or type `flybase_gene_id` / FBgn values. Do not copy IDs from memory, chat, FlyBase pages, or an old spreadsheet. Do not "fix" a dash by filling an ID. Edit `validated_*.csv` or FBgn columns **only if the user explicitly asks**.
> After conversion, review the xlsx (each FBgn is a hyperlink). If anything is unmatched, paste those symbols at `https://flybase.org/convert/id`. Stocker runs **only** on `validated_*.csv`, and **only after that review**.

A. Copy the user's list unchanged. One new folder per request. Do not edit the user's CSV. If you started from Excel, export a CSV and keep the workbook.

```
python3 -c "from pathlib import Path; Path(
    'data/gene_sets/YourName-Project-MMDDYYYY'
).mkdir(parents=True, exist_ok=True)"
```

One gene set = one CSV. First row must be exactly `ext_gene`. One Drosophila gene symbol per data row. No empty header, no extra title rows, no human gene symbols mixed in unless you intend to leave those unmatched.

B. Convert symbols to FBgn IDs. Agents run this script. They do not fill IDs.

```
python3 scripts/fetch_fbgn_ids.py "data/gene_sets/YourName-Project-MMDDYYYY"
```

Reads source `*.csv` files (skips its own sidecars). Leaves the original file bytes unchanged. Writes:

```
validated_<original>.csv     mapped rows only (FBgn IDs present)
needs-review.csv             unmatched rows from every input + source_file
validated_<original>.xlsx    review workbook: Validated + Needs review sheets
```

In the xlsx, each `flybase_gene_id` is a hyperlink to a `https://flybase.org/reports/FBgn0023076`-style report. **Tell the requester this xlsx exists and wait for them to review it for precision.** Optional second argument is the symbol column (default `ext_gene`).

C. Human review gate. Stop here until the list is confirmed.

Open `validated_<original>.xlsx`. Click every FBgn. Confirm it is the intended current *D. melanogaster* gene, not a paralog, a human gene, or an old secondary ID that now points elsewhere.

If `needs_review.csv` has any data rows, paste those symbols at `https://flybase.org/convert/id` before anyone treats the list as ready. Enable "Match synonyms / secondary IDs" if a current symbol fails.

Typical unmatched causes (fix on a **copy** of the user's list, then re-run B; never edit the original in place):

- RNA-seq uniquifiers (`sky1`, `RpL231`) – put the parent symbol on a new working CSV, then re-run the script. Do not strip a digit that is part of the real name (`Argk1`, `miple1`).
- Not a fly gene (for example `EGFP`) – leave it in needs-review; do not invent an ID.
- Old CG numbers / synonyms – the script usually fills `primary_symbol`. That is expected.
- Mixed species or outdated symbols – resolve on FlyBase, then put the current fly symbol on the working CSV.

Do not start Stocker while a real fly gene remains unmatched. If FlyBase disagrees with the script, keep the script output, flag the row, and ask the maintainer.

4. Default config – what the knobs mean

Unless the requester names another JSON file, use `data/config/stock_split_config_priority_example.json`. This file is the sorting protocol, not a data source. It does not add genes or stocks. It only tells Stocker which columns to read, what "topic-relevant" means, which phenotype buckets to fill, and how many stocks to keep.

Do not swap this file for `stock_split_config_example.json` (publication-evidence Ref matrix, effectively no stock caps) or `stock_split_config_priority_all_phenotypes.json` (exhaustive coverage) unless that is what was asked for. Those files change both the sheets and the limits.

Settings in the default file, in plain English:

Setting	Value and meaning
`input.geneCol` `input.inputGeneCol`	`flybase_gene_id` and `ext_gene`. Stocker looks up reagents by FBgn and keeps your original symbol for display.
`input.skipFbgnidConversion`	**true**. Stage 1 trusts the validated CSV IDs. That is why section 3 exists: conversion already happened outside Stocker. Do not set this false and also skip review.
`relevantSearchTerms`	**sleep, circadian, rhythm, locomotor** (case-insensitive). A paper is topic-relevant if its title or abstract contains one of these words. That is a text match, not a claim that the paper demonstrated the phenotype.
`phenotypeSimilarityTargets`	The same four words, each with an embedding phrase. Used for curated-phenotype tiers (Phenotype++ if a FlyBase phenotype name matches a target; Phenotype+ if any curated phenotype exists) and for optional cosine columns when embeddings run.
`maxStocksPerGene` `maxStocksPerAllele`	**5** and **3**. See section 5. These are the shortlist caps.
`pubmed.batchSize`	**50**. How many PubMed records are requested per batch. Does not change which papers count.

`embeddings.enabled`	**true** in this file. Adds cosine-similarity columns against the phenotype targets. Needs `OPENAI_API_KEY` for those columns. Leave it as shipped unless asked to change it.
`validation.enabled`	**false**. GPT does not read full text to decide whether a knockdown "worked." Do not turn this on unless the requester asks.
`output.preserveUnsplitWorkbook`	**false**. The primary run keeps the organized workbook only, not the intermediate Stage 1 file.

Publication-evidence tiers (computed as `ALLELE_PAPER_RELEVANCE_SCORE`, inherited by every stock of that allele):

Tier	Meaning
Ref++ (score 2)	At least one linked paper mentions a configured keyword in the title or abstract.
Ref+ (score 1)	The allele has papers, but none of those titles or abstracts hit the keywords.
Ref- (score 0)	FlyBase lists no publications for the allele.

This default does **not** split sheets by Ref++ / Ref+ / Ref-. It keeps stocks that have **any curated FlyBase phenotype** (Phenotype+ or Phenotype++), then bins them into six named buckets. Combinations are tested top to bottom. Each (`stock_id`, collection) reagent lands in only the first matching sheet.

Sheet (tab name)	Who gets in
1 BDSC·RNAi·Pheno	Bloomington RNAi reagent with a curated phenotype.
2 VDRC·RNAi·Pheno	Vienna / VDRC RNAi reagent with a curated phenotype.
3 BDSC·Allele·Pheno	Bloomington allele or insertion (not the broad RNAi/UAS proxy) with a phenotype.
4 OtherSC·Allele·Pheno	Any other orderable stock-center allele/insertion with a phenotype.
5 Custom·RNAi·Pheno	Phenotype-backed RNAi with no orderable FBst record.
6 Custom·Allele·Pheno	Phenotype-backed allele/insertion with no orderable FBst record.

Empty categories do not appear as tabs. A Bloomington RNAi stock that also matches the custom RNAi rule appears only on sheet 1. Changing sheet order changes which stocks spend the per-gene quota.

Useful filter meanings in this file: **RNAi_reagent** is FlyBase `transgenic_product_class_terms` containing `RNAi_reagent`. **AlleleOrInsertion** is the complement of the repo's broad UAS/RNAi proxy (genotype has UAS or RNAi, or a VDRC GD/KK/VSH knockdown). **Stock Center** vs **Non Stock Center** is whether an orderable FBst record exists. **Phenotype** is any non-zero curated-phenotype score.

5. Stock limits and how sheets are filled

After identity, reagent expansion, and first-match sorting, the default config keeps a shortlist. These two numbers are the ones people miss:

Limit	What it does in the default file
`maxStocksPerGene` = 5	Once a gene already has 5 accepted stocks anywhere in the workbook, later matching stocks of that gene are dropped.

`maxStocksPerAllele` = 3	Once an allele already has 3 accepted stocks, further stocks of that same allele are dropped even if the gene still has room.

How the quota is spent:

1. Stocks are sorted so stronger publication evidence is considered first (Ref++ before Ref+, then more keyword-hitting papers, then more total papers).
2. Sheet 1 is filled first, then sheet 2, and so on. A stock accepted on an earlier sheet never appears again.
3. Gene and allele counters are **shared across all six sheets**. Five Bloomington RNAi stocks for *Clk* on sheet 1 means later sheets get no further *Clk* stocks.
4. A stock that matched sheet 1 but was rejected because the gene or allele was already at the cap is not spilled onto sheet 2. Leftovers are omitted.
5. The Contents sheet and `combination_counts_summary.xlsx` count what survived these limits, not every FlyBase stock that theoretically matched.

Experimentally: you receive a prioritized shortlist, not an inventory. If the requester wants every phenotype-backed reagent, they must name a different config (the exhaustive `stock_split_config_priority_all_phenotypes.json` file lifts these caps) or raise `maxStocksPerGene` / `maxStocksPerAllele` on a copy of the default JSON. Do not silently lift the caps.

Limits do not check whether BDSC or VDRC currently ships the stock. They only cap how many FlyBase records are written into the workbook.

6. Run Stocker on the validated list

Run only after section 3 is done. Stay with `data/config/stock_split_config_priority_example.json` unless another JSON file was named. Discovery skips the original CSV and `needs-review.csv` when `validated_*.csv` is present.

```
python3 -m fl_ai_reagent_stocker run \
  "data/gene_sets/YourName-Project-MMDDYYYY" \
  --config data/config/stock_split_config_priority_example.json
```

Optional recount after a partial rerun:

```
python3 scripts/summarize_combination_counts.py \
  "data/gene_sets/YourName-Project-MMDDYYYY" \
  --config data/config/stock_split_config_priority_example.json
```

7. What you should have when it finishes

Keep the original CSV, the validated CSV, the review xlsx, and needs-review.csv. They are the record of what was queried and what was checked.

```
data/gene_sets/<run>/<original>.csv                          (untouched user list)
data/gene_sets/<run>/validated_<original>.csv                (stocker input)
data/gene_sets/<run>/validated_<original>.xlsx               (human ID review)
data/gene_sets/<run>/needs-review.csv
data/gene_sets/<run>/Per Gene Set Runs/<validated name>/
    Stocks/<validated name>_aggregated.xlsx
data/gene_sets/<run>/combination_counts_summary.xlsx
```

Share the aggregated workbook and the counts table. Inside the workbook: **Contents**, one tab per populated category from section 4, **References** (papers cited by stocks that made it through the limits), and **Stock Sheet by Gene**. Cosine columns appear when embeddings ran with a key. Check BDSC, VDRC, or the relevant center before ordering.

8. If something fails

Symptom	What to do
No project on disk	Install from GitHub (section 2). Source: https://github.com/aadish98/fl.AI-reagent-stocker. A code-only zip still needs `refresh_flybase_data.py` so `entity_publication` is present.
git clone fails (private repo)	Ask the maintainer for GitHub access, or take a complete folder from a shared drive and skip clone.
Run dies looking up papers / missing entity_publication	The citation table is not in git. From the repo root: `python3 scripts/refresh_flybase_data.py`.
"No gene-list CSV files found"	Wrong path, or files are not named .csv. Run from the repo root.
"Missing required columns"	Stocker needs `ext_gene` and `flybase_gene_id` on the validated CSV. Re-run conversion; do not type IDs.
needs-review.csv has rows	Do not run Stocker. Review the xlsx and https://flybase.org/convert/id. Fix symbols on a new working CSV, then re-run the script.
Workbook has far fewer stocks than FlyBase shows	Expected under the default caps (5 / gene, 3 / allele) and first-match sheets. See section 5. Do not lift limits unless asked.
python: command not found	Use `python3` or `python3 -m fl_ai_reagent_stocker` so the same interpreter is used.
Embedding / OpenAI errors	Stock sheets can still be used. Cosine columns need `OPENAI_API_KEY` in `.env`. Do not turn on validation to "fix" this.

9. Agent command policy

- Run only the commands in this SOP, from the Stocker repo root.
- Never edit the user-generated gene list. Never write FBgn IDs by hand. Always call `scripts/fetch_fbgn_ids.py`.
- Tell the user that `validated_*.xlsx` exists and wait for them to review it. If `needs-review.csv` is non-empty, send them to `https://flybase.org/convert/id` first.
- Run Stocker only on `validated_*.csv`, and only after that review.
- Do not edit `validated_*.csv` or FBgn columns unless the user explicitly asks.
- Use `data/config/stock_split_config_priority_example.json` unless another JSON file is named. Do not lift stock limits, reorder sheets, or turn on GPT validation unless asked. Leave embeddings as shipped.
- Do not invent stocks or paper conclusions. If a command fails, show the error. Do not invent a workaround that writes gene IDs.

Optional further reading in the repo: `docs/how_stocker_works.md`, `docs/pipeline_usage.md`, `docs/stock_split_config_priority_example.md`, `docs/AGENTS.md`.